Static Analysis Benchmark Report

1. Dataset Description

Dataset Overview

The benchmark dataset consists of a curated set of vulnerable projects used for evaluating static analysis tools. Based on the archive structure:

- Root directory: Benchmark/
- Contains:
  - Analysis scripts (analysing_scripts/)
  - Tool reports (reports/)

2. Tools Used

Analyzers

The following static analysis tools were used:

- PMD (Java only)
- PVS-Studio (Java only)
- SonarQube
- Semgrep
- CodeQL
- OpenGrep
- Joern

OWASP BenchmarkJava

The OWASP BenchmarkJava dataset is a mature and widely used benchmark for evaluating static application security testing (SAST) tools. In version 1.2, it contains approximately 2,740 test cases, each implemented as an individual Java servlet. Earlier versions contained significantly more test cases (over 20,000), but the current version focuses on a curated and balanced subset.

Each test case represents a single vulnerability instance (or a false positive case) associated with a specific CWE category. The dataset is structured as a full web application, with the main source code located in the `src/` directory. It also includes supporting scripts, tooling, and a ground truth file (`expectedresults-1.2.csv`) used for evaluation.

The project is primarily written in Java, with some HTML components used for web interaction. In practice, this results in thousands of Java classes and a codebase that reaches tens of thousands of lines of code.

The dataset covers a focused set of common web vulnerabilities. The main CWE categories include:

- CWE-78 (Command Injection)
- CWE-89 (SQL Injection)

- CWE-79 (Cross-Site Scripting, XSS)
- CWE-22 (Path Traversal)
- CWE-90 (LDAP Injection)
- CWE-643 (XPath Injection)
- CWE-327 (Use of Broken or Risky Cryptographic Algorithm)
- CWE-328 (Weak Hashing)
- CWE-330 (Insufficiently Random Values)
- CWE-501 (Trust Boundary Violation)
- CWE-614 (Sensitive Cookie Without Secure Flag)

Overall, BenchmarkJava provides strong coverage of classical web application vulnerabilities and is considered a standard dataset for SAST benchmarking.

OWASP BenchmarkPython

The OWASP BenchmarkPython dataset is a newer and less mature benchmark compared to its Java counterpart. It contains approximately 1,230 test cases and is currently considered a preliminary version (v0.1).

Like the Java version, each test case represents a single vulnerability or a negative (non-vulnerable) example. The dataset is also structured as a web application, but it is significantly smaller in size—roughly two to three times smaller than BenchmarkJava in terms of both test cases and overall code volume.

The project is written primarily in Python and follows a similar philosophy: synthetic, well-isolated test cases designed for precise evaluation of static analyzers.

BenchmarkPython covers a broader and slightly more modern range of CWE categories compared to the Java dataset. These include:

- CWE-78 (Command Injection)
- CWE-89 (SQL Injection)
- CWE-79 (Cross-Site Scripting, XSS)
- CWE-22 (Path Traversal)
- CWE-90 (LDAP Injection)
- CWE-94 (Code Injection)
- CWE-502 (Deserialization of Untrusted Data)
- CWE-601 (Open Redirect)
- CWE-611 (XML External Entity, XXE)
- CWE-643 (XPath Injection)
- CWE-328 (Weak Hashing)
- CWE-330 (Weak Randomness)
- CWE-501 (Trust Boundary Violation)
- CWE-614 (Secure Cookie Issues)

Notably, this dataset includes vulnerability types that are less represented in the Java benchmark, such as deserialization issues and XXE.

Comparison and Key Takeaways

BenchmarkJava is larger, more mature, and more widely adopted. It provides a stable and well-understood baseline for evaluating static analysis tools, especially for traditional web vulnerabilities.

BenchmarkPython, while smaller and less mature, introduces a broader set of CWE categories and reflects more modern vulnerability patterns. However, its limited size and early-stage development make it less comprehensive for large-scale evaluation.

Both datasets share the same core design principles:

- Synthetic, controlled test cases
- One vulnerability per test
- Explicit ground truth labeling

These characteristics make them particularly suitable for quantitative evaluation of static analysis tools using metrics such as precision, recall, and F1-score.

Report:

- Total report files:
  - Python reports: 5
  - Java reports: 7
- Total analysis scripts: 7
- Dataset includes projects in:
  - Python
  - Java

3. Scanning Methodology

All the project were scanned by `Whole-project analysis` method. It is the only available aproach for owasp benchmarks since these projects are too huge to scan file-by-file manually.

Reproducibility Scripts

The my analysis includes custom scripts:

- my_pmd_results.py
- my_pvs-studio_results.py
- my_sonar_results.py
- my_semgrep_results.py
- my_codeql_results.py
- my_opengrep_results.py
- my_joern_results.py

These scripts:

1. Parse raw tool outputs
2. Normalize findings

3. Map findings to CWE IDs
4. Compare results against ground truth
5. Compute evaluation metrics